

40009 311

Kotlin Journey Planning

Submitters

kw1425

12
—
20

Emarking

```
1: Final Tests: Summary for kw1425 of c1
2: PPT 21
3: -----
4:
5:   Public Tests:
6:     Compiles:      1 / 1
7:     Tests Pass:    1 / 1
8:     Style Check:   1 / 1
9:
10: Git Repo: git@gitlab.doc.ic.ac.uk:lab2526_autumn/kotlinjourneyplanner_kw1425.git
11: Commit ID: 235ee
12: Git Diff: https://gitlab.doc.ic.ac.uk/lab2526_autumn/kotlinjourneyplanner_kw1425/-/compare/master..235ee?from_project_id=134876
```

Final Tests	RoutePlanner.kt: 1/2	Katie Wan(kw1425):c1:21	Final Tests	RoutePlanner.kt: 2/2	Katie Wan(kw1425):c1:21
	<pre> 1: package journeyplan 2: 3: // import com.sun.jdi.connect.spi.Connection 4: // import journeyplan.Segment 5: 6: // Add your code for the route planner in this file. 7: val northernLine = Line("Northern") 8: val victoriaLine = Line("Victoria") 9: val centralLine = Line("Central") 10: 11: val highgate = Station("Highgate") 12: val archway = Station("Archway") 13: val tufnellPark = Station("Tufnell Park") 14: val kentishTown = Station("Kentish Town") 15: val camden = Station("Camden Town") 16: val euston = Station("Euston") 17: val warrenStreet = Station("Warren Street") 18: val oxfordCircus = Station("Oxford Circus") 19: val bondStreet = Station("Bond Street") 20: 21: class SubwayMap(22: val segments: List<Segment> 23:) { 24: fun routesFrom(25: origin: Station, 26: destination: Station 27:): List<Route> = buildRoutes(origin, destination, emptyList<Station>()) 28: 29: private fun buildRoutes(30: origin: Station, 31: destination: Station, 32: visited: List<Station> 33:): List<Route> { 34: val connections = segments.filter { it.s1 == origin && !visited.contains(it.s2) } 35: val returnList = mutableListOf<Route>() 36: for (connection in connections) { 37: if (connection.s2 == destination) { 38: returnList += Route(listOf(connection)) 39: } else { 40: val subRoutes = 41: buildRoutes(connection.s2, destination, visited + origin) 42: val extendedRoutes = 43: subRoutes.map { subRoute -> 44: Route(listOf(connection) + subRoute.segments) 45: } 46: returnList += extendedRoutes 47: } 48: } 49: return returnList 50: } 51: } 52: 53: class Route(54: val segments: List<Segment> 55:) { 56: override fun toString(): String { 57: var origin = segments.first().s1.toString() 58: val destination = segments.last().s2.toString() 59: val duration = duration() 60: val changes = numChanges() 61: returnString = "\$origin to \$destination - \$duration minutes, \$changes changes\n" 62: var line = segments.first().line.toString() 63: for (segment in segments) { 64: val s1 = segment.s1.toString() </pre>	1/2 → add a function to sort routes by as a parameter 2-4 you don't check for closed stations or suspended lines → use a StringBuilder here		<pre> 65: val newline = segment.line.toString() 66: if (newline != line) { 67: returnString += " - \$origin to \$s1 by \$line\n" 68: line = newline 69: origin = s1 70: } 71: } 72: returnString += " - \$origin to \$destination by \$line\n" 73: return returnString.trimEnd().trimIndent() 74: } 75: 76: fun numChanges(): Int { 77: var result = 0 78: if (segments.isEmpty()) return result 79: var curr = segments.first().line 80: for (segment in segments) { 81: if (segment.line != curr) { 82: curr = segment.line 83: result++ 84: } 85: } 86: return result 87: } 88: 89: fun duration(): Int { 90: var result = 0 91: for (segment in segments) { 92: result += segment.time 93: } 94: return result 95: } 96: } 97: 98: fun londonUnderground(): SubwayMap = 99: SubwayMap(100: listOf(101: Segment(highgate, archway, northernLine, 3), 102: Segment(archway, kentishTown, northernLine, 3), 103: Segment(kentishTown, camden, northernLine, 3), 104: Segment(camden, euston, northernLine, 3), 105: Segment(euston, warrenStreet, victoriaLine, 3), 106: Segment(warrenStreet, oxfordCircus, victoriaLine, 3), 107: Segment(tufnellPark, archway, northernLine, 3), 108: Segment(archway, highgate, northernLine, 3), 109: Segment(oxfordCircus, bondStreet, centralLine, 2) 110:) 111:) 112: 113: fun main() { 114: val map = londonUnderground() 115: println(map.routesFrom(tufnellPark, highgate).toString()) 116: } </pre>	try using a higher-order function here. 2/2 1/2 → use sumOf

```
1: package journeyplan
2:
3: // Add your code for modelling public transport networks in this file.
4: class Station(
5:     val name: String
6: ) {
7:     override fun toString(): String = name
8: }
9:
10: class Line(
11:     val name: String
12: ) {
13:     override fun toString(): String = "$name Line"
14: }
15:
16: class Segment(
17:     val s1: Station,
18:     val s2: Station,
19:     val line: Line,
20:     val time: Int
21: )
```

1/2

1/2

1/1

Final Tests	RoutePlannerTest.kt: 1/2	Katie Wan(kw1425):c1:21	Final Tests	RoutePlannerTest.kt: 2/2	Katie Wan(kw1425):c1:21
--------------------	---------------------------------	--------------------------------	--------------------	---------------------------------	--------------------------------


```

1: package journeyplan
2:
3: import org.junit.Test
4: import kotlin.test.assertEquals
5:
6: class RoutePlannerTest {
7:     val northernLine = Line("Northern")
8:     val victoriaLine = Line("Victoria")
9:     val centralLine = Line("Central")
10:
11:     val highgate = Station("Highgate")
12:     val archway = Station("Archway")
13:     val tufnellPark = Station("Tufnell Park")
14:     val kentishTown = Station("Kentish Town")
15:     val camden = Station("Camden Town")
16:     val euston = Station("Euston")
17:     val warrenStreet = Station("Warren Street")
18:     val oxfordCircus = Station("Oxford Circus")
19:     val bondStreet = Station("Bond Street")
20:
21:     val tufnellParkToHighgate =
22:         Route(
23:             listOf(
24:                 Segment(tufnellPark, archway, northernLine, 3),
25:                 Segment(archway, highgate, northernLine, 3)
26:             )
27:         )
28:
29:     val highgateToOxfordCircus =
30:         Route(
31:             listOf(
32:                 Segment(highgate, archway, northernLine, 3),
33:                 Segment(archway, kentishTown, northernLine, 3),
34:                 Segment(kentishTown, camden, northernLine, 3),
35:                 Segment(camden, euston, northernLine, 3),
36:                 Segment(euston, warrenStreet, victoriaLine, 3),
37:                 Segment(warrenStreet, oxfordCircus, victoriaLine, 3)
38:             )
39:         )
40:
41:     val camdenToBondStreet =
42:         Route(
43:             listOf(
44:                 Segment(camden, euston, northernLine, 3),
45:                 Segment(euston, warrenStreet, victoriaLine, 3),
46:                 Segment(warrenStreet, oxfordCircus, victoriaLine, 3),
47:                 Segment(oxfordCircus, bondStreet, centralLine, 2)
48:             )
49:         )
50:
51:     val highgateToHighgate =
52:         Route(emptyList())
53:
54:     @Test
55:     fun `can calculate number of changes`() {
56:         assertEquals(0, tufnellParkToHighgate.numChanges())
57:         assertEquals(1, highgateToOxfordCircus.numChanges())
58:         assertEquals(2, camdenToBondStreet.numChanges())
59:     }
60:
61:     @Test
62:     fun `can calculate total duration`() {
63:         assertEquals(6, tufnellParkToHighgate.duration())
64:         assertEquals(18, highgateToOxfordCircus.duration())
65:         assertEquals(11, camdenToBondStreet.duration())
66:     }

```

```

67:
68:     @Test
69:     fun `toString omits calling points`() {
70:         assertEquals(
71:             """
72:             Tufnell Park to Highgate - 6 minutes, 0 changes
73:             - Tufnell Park to Highgate by Northern Line
74:             """.trimIndent(),
75:             tufnellParkToHighgate.toString()
76:         )
77:     }
78:
79:     @Test
80:     fun `toString shows changes`() {
81:         assertEquals(
82:             """
83:             Highgate to Oxford Circus - 18 minutes, 1 changes
84:             - Highgate to Euston by Northern Line
85:             - Euston to Oxford Circus by Victoria Line
86:             """.trimIndent(),
87:             highgateToOxfordCircus.toString()
88:         )
89:     }
90:
91:     @Test
92:     fun `origin same as destination changes`() {
93:         assertEquals(0, highgateToHighgate.numChanges())
94:     }
95:
96:     @Test
97:     fun `origin same as destination duration`() {
98:         assertEquals(0, highgateToHighgate.duration())
99:     }
100: }

```

```
1: package journeyplan
2:
3: import org.junit.Test
4: import kotlin.test.assertEquals
5:
6: class TravelModelTest {
7:     @Test
8:     fun `printing stations shows their names`() {
9:         assertEquals("South Kensington", Station("South Kensington").toString())
10:        assertEquals("Knightsbridge", Station("Knightsbridge").toString())
11:    }
12:
13:     @Test
14:     fun `printing lines shows their names`() {
15:         assertEquals("District Line", Line("District").toString())
16:         assertEquals("Circle Line", Line("Circle").toString())
17:     }
18: }
```

Final Tests

testResults.txt: 1/1

Katie Wan(kw1425):c1:21

```
1: ----- Test Output -----
2: Running LabTS build... (Fri 21 Nov 17:44:51 UTC 2025)
3:
4: Submission summary...
5: You made 2 commits
6:   - e12559e finish [4 files changed, 237 insertions, 95 deletions]
7:   - 235ee3d finish [1 file changed, 2 insertions, 2 deletions]
8:
9: Preparing...
10:
11: BUILD SUCCESSFUL in 1s
12:
13: Compiling...
14: BUILD SUCCESSFUL in 16s
15:
16: Running tests...
17:
18: RoutePlannerTest > origin same as destination duration PASSED
19:
20: RoutePlannerTest > toString shows changes PASSED
21:
22: RoutePlannerTest > origin same as destination changes PASSED
23:
24: RoutePlannerTest > toString omits calling points PASSED
25:
26: RoutePlannerTest > can calculate total duration PASSED
27:
28: RoutePlannerTest > can calculate number of changes PASSED
29:
30: TravelModelTest > printing lines shows their names PASSED
31:
32: TravelModelTest > printing stations shows their names PASSED
33:
34: BUILD SUCCESSFUL in 2s
35:
36: Checking code style...
37: BUILD SUCCESSFUL in 2s
38: Finished auto test. (Fri 21 Nov 17:46:01 UTC 2025)
39:
40: ----- Test Errors -----
41:
```